Sessió 7

Desenvolupament Web Full Stack (React, Node, Mongo)
Curs 2025-2026

Pràctica: E-commerce. Sessió 7

1. Dates....3
2. Objectius....3
3. Feina prèvia abans de la sessió....3
4. Feina durant la sessió i abans de la següent sessió....4
4.1 Instal·lar bcrypt....4
4.2 Instal·lar JWT....4
4.3 Implementar Registre i Login d'usuaris amb hashing de contrasenyes....4
4.4 Crear l'esquema "Usuari.js" amb hashing ('models/Usuari.js')....5
4.5 Controlador "controllers/usuariController.js"....5
4.6 Service "service/usuariService.js"....6
S'encarrega d'interactuar amb la base de dades, xifrar contrasenyes, generar tokens, etc....6
4.7 Routes "routes/usuariRoutes.js"....6
4.8 Executar consultes Postman....7
4.9 Comprovar MongoDB....8
Referències....9
Apèndixs....9
A. Comandes útils de Node/Nodemon....9
B. Fitxer .gitignore recomanat....9
C. Validacions habituals en Mongoose....9
D. Errors comuns i depuració....9
Lliurable de la Sessió 7....10

1. Dates

- 12/11 - 19/11

2. Objectius

Pes	Preferent?	Funcionalitat
0,4	✓	Usuaris: Registre i Login
0,4	✓	Hash Encrypt (bcrypt) i JWT (Tokens)
0,2	✓	Comprovar Registre/Login Postman/MongoDB

3. Feina prèvia abans de la sessió

- Llegir sobre **bcrypt i JWT (JSON Web Token)**

3.1 Què és bcrypt?

bcrypt és una llibreria de hashing de contrasenyes. És un algorisme adaptatiu dissenyat per a convertir contrasenyes en hashes xifrats de manera segura.

Per a què usar bcrypt?

- Seguretat: Les contrasenyes en text pla poden ser robades i usades per atacants. bcrypt xifrarà les contrasenyes, i si algú accedeix a la base de dades, només veurà hashes, no les contrasenyes reals.
- Resistència a atacs de força bruta: bcrypt inclou un factor de "sal" i un mecanisme per a fer el càlcul més lent, dificultant que els hackers puguin endevinar les contrasenyes amb atacs de força bruta.

- Sal: una cadena aleatòria afegida per a evitar atacs de taules de precomputació (rainbow tables).

3.2 Què és JWT (JSON Web Token)?

JWT és un estàndard obert (RFC 7519) per a crear tokens segurs en format JSON que s'usen per a autenticar i autoritzar els usuaris en aplicacions web, APIs, etc.

Per a què serveix JWT?

- Autenticació: Quan un usuari inicia sessió, el servidor li retorna un token JWT que conté informació xifrada sobre l'usuari.
- Autorització: En sol·licituds futures, aquest token s'envia (normalment en un header) per a verificar qui és l'usuari i quins permisos té.
- Sessions sense estat (stateless): No necessites mantenir sessions en el servidor; el token porta tota la informació necessària.

4. Feina durant la sessió i abans de la següent sessió

4.1 Instal·lar bcrypt

npm install bcrypt

4.2 Instal·lar JWT

npm install jsonwebtoken

4.3 Implementar Registre i Login d'usuaris amb hashing de contrasenyes

- Registre: procés on l'usuari crea un compte, enviant dades com a email, contrasenya, nom, etc.
- Login: procés on un usuari existent introdueix les seves credencials per a autenticar-se.

- Hashing de Contrasenya: convertir la contrasenya en una cadena xifrada abans d'emmagatzemar-la en la base de dades, per a major seguretat. **Es recomana usar la llibreria bcrypt.**

4.4 Crear l'esquema "Usuari.js" amb hashing ('models/Usuari.js')

```
const bcrypt = require('bcrypt');
```

…..

```
// Abans de guardar, encriptar la contrasenya
usuariSchema.pre('save', async function(next) {
    if (!this.isModified('password')) return next();
    const salt = await bcrypt.genSalt(10);
    this.password = await bcrypt.hash(this.password, salt);
    next();
})
```

……

4.5 Controlador "controllers/usuariController.js"

Només maneja les sol·licituds (registre, login) i delega la lògica a un servei.

```
api > src > controllers > JS usuariController.js > ...
const Usuari = require('../models/Usuari');

const usuariService = require('../services/usuariService');

const registrarse = async (req, res) => {
  try {
    const usuariCreado = await usuariService.registrarUsuari(req.body);
    res.status(201).json({ ok: true, data: usuariCreado });
  } catch (error) {
```

…..

4.6 Service “services/usuariService.js”

S'encarrega d'interactuar amb la base de dades, xifrar contrasenyes, generar tokens, etc.

```
api > src > services > JS usuariServices.js > ...
const Usuari = require('../models/Usuari');

const bcrypt = require('bcrypt');
const jwt = require('jsonwebtoken');

// Funció per a registrar usuari
const registrarUsuari = async ({ nom, email, password }) => {
  // Validar si l'email ja existeix
  const existe = await Usuari.findOne({ email });
  if (existe) throw new Error('Email ja està en ús');

```

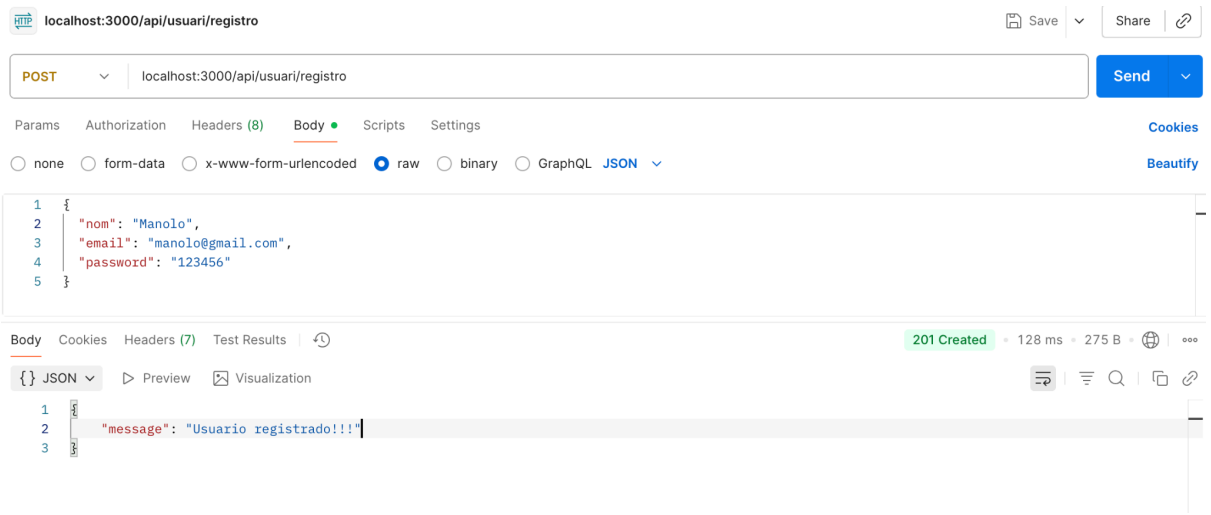
…….

4.7 Routes “routes/usuariRoutes.js”

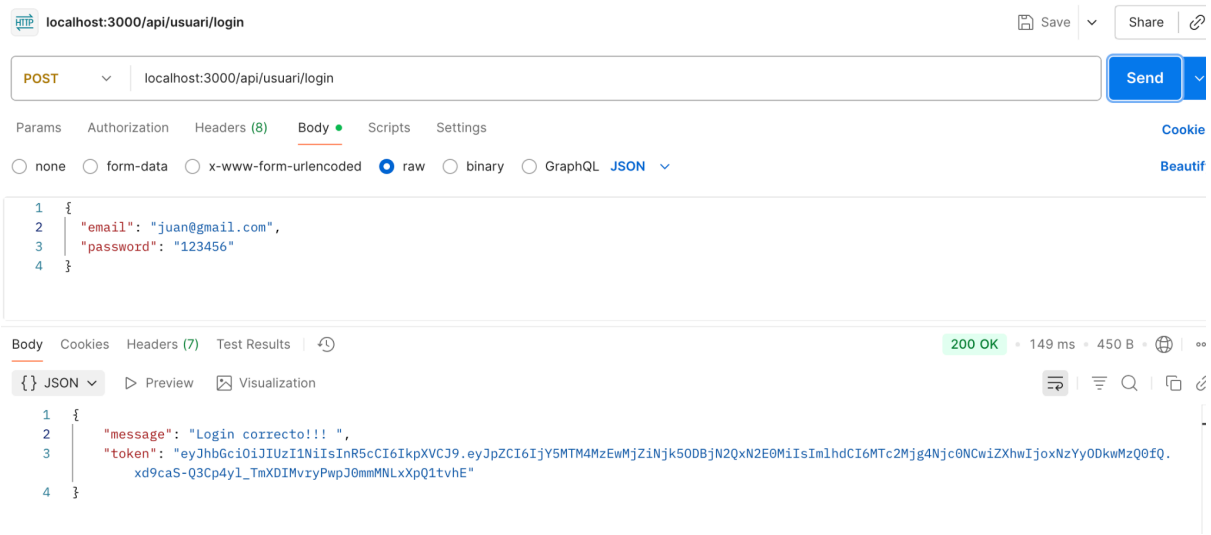
```
api > src > routes > JS usuariRoutes.js > ...
const express = require('express');
const routerUsuari = express.Router();
const usuariController = require('../controllers/usuariController');

routerUsuari.post('/registro', usuariController.registrarse);
routerUsuari.post('/login', usuariController.login);

module.exports = routerUsuari;
```

4.8 Executar consultes Postman

Registre:

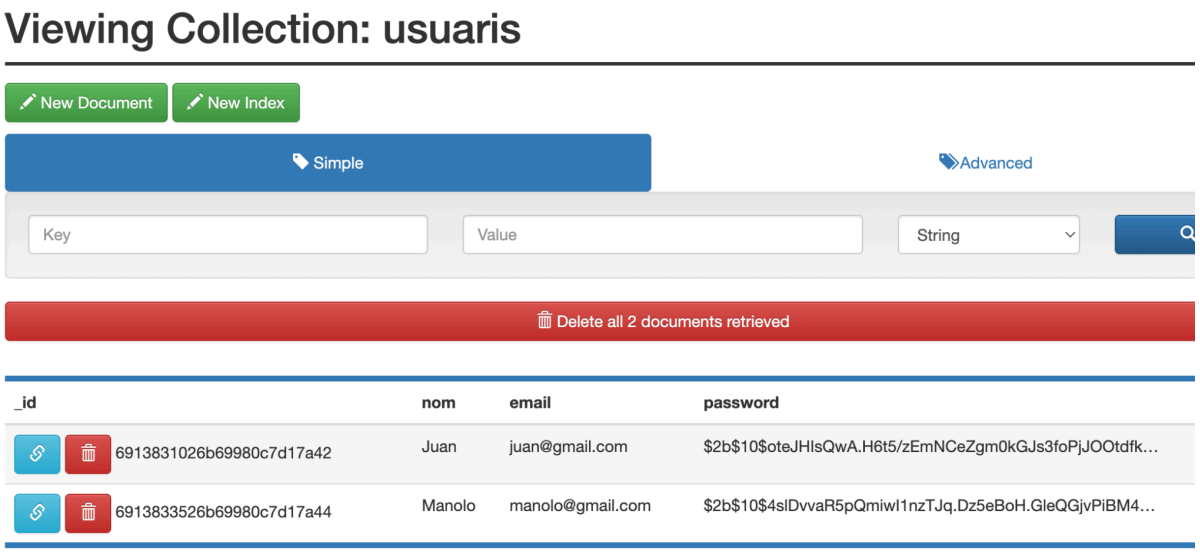


Login:



4.9 Comprovar MongoDB

Mostra el resultat amb captures de pantalla de les execucions de Registrar Usuari en la teva base de dades en MongoDB



Referències

- What is an ADR? – Michael Nygard
- Awesome ADR – repositori amb exemples
- MongoDB – Documentació oficial
- Draw.io (eina de diagrames gratuïta)

Apèndixs

A. Comandes útils de Node/Nodemon

- `npm run dev` → iniciar en mode desenvolupament.
- `CTRL+C` → aturar servidor.
- Escriure de forma clara i breu, no més d'1 pàgina.

B. Fitxer `.gitignore` recomanat

```
node_modules/

dist/

.env
```

C. Validacions habituals en Mongoose

- `required`
- `minlength`
- `maxlength`
- `enum`
- `unique`
- `match`.

D. Errors comuns i depuració

- No carregar `.env` → MONGO_URI indefinit.
- Port 3000 ocupat → cal canviar `PORT` al `.env`.
- Índexs duplicats si es fan canvis freqüents al model.

Lliurable de la Sessió 7

PDF amb totes les explicacions i link a GitHub.